Assignment 2: HTTP API, gRPC and Metrics

Invoice Job Queue

Enterprise Software Development, Fall 2026

Myrah Hussain

Repository: github.com/myrahhussain/ESD-Assignments

Overview

This report builds on Assignment 1 (a Flask job queue with Prometheus, Grafana, Filebeat, Elasticsearch and Kibana). The shop owner sends an order, a background worker makes a PDF invoice, and anyone can check the job status. Assignment 2 does three things. It cleans up the HTTP API (Part A), adds a gRPC service that runs as its own process and shares the same database and business rules (Part B), and adds metrics for both so they can be compared on one dashboard (Part C). All commands to run and test everything are in `README.md`.

Part A: The HTTP API

A1. REST constraints

Constraint	How the API handles it	Status
Client-server	The HTTP app, the gRPC app and the callers (Postman, `caller.py`) are separate programs. A client only needs the URLs and the JSON shape.	Met
Stateless	Each request carries everything the server needs (the body and the optional `Idempotency-Key` header). Job state lives in the shared database, not in a session. One exception: the rate limiter keeps its counts in memory per client IP.	Mostly met
Cacheable	Not used on purpose. A job's status changes over time, so a cached answer would be wrong. A `Cache-Control: no-store` header on `GET /jobs/{id}` is the clear next step.	Not used
Layered system	A client only talks to the API. It cannot tell if a proxy or load balancer sits in front. Prometheus and Filebeat sit beside the app, not in the request path. No proxy is used today.	Met by design
Uniform interface	Resources are `/jobs` and `/jobs/{job_id}`. POST creates, GET reads. Standard status codes are used (201, 200, 400, 404, 429, 500). Every error has one shape. New jobs return a `Location` header.	Met
Code on demand (optional)	The API returns data only, never code.	Not used

A2. Endpoint clean-up and the before / after table

`/cardinality-test` and `/cardinality-test-safe` are demo endpoints from Assignment 1. They are kept, listed in `openapi.yaml` and described as demos. The `/chaos` endpoint is kept because the reliability tests use it, but it now checks its input. Table 1 compares the API before and after.

Table 1. API before (Assignment 1) and after (Assignment 2).

Area	Assignment 1	Assignment 2
Create a job	Returns `200` with no `Location` header.	Returns `201 Created` with a `Location: /jobs/{id}` header.
Bad request body	A missing field raised an unhandled error and the server answered `500`.	Returns `400` with a message that names the problem, for example `Field 'items' is required and must be a non-empty list.`
Unknown job	`404` with `{"error": "Job not found"}`.	`404` with `{"error": {"code": "job_not_found", "message": "..."}}`.
Error format	Different shapes in different places.	One envelope for every error: `{"error": {"code", "message"}}`.
Sending the same order twice	Made a duplicate job each time.	The same `Idempotency-Key` returns the original job (`200` and `Idempotent-Replayed: true`). No second job is made.

Area	Assignment 1	Assignment 2
Too many requests	No limit.	5 requests per 60 seconds per client. Extra requests get `429` with a `Retry-After` header.
Where jobs live	An in-memory dictionary, lost on restart.	SQLite (`jobs.db`), shared by the HTTP and gRPC programs.
`/chaos` input	No checks.	Checked. Bad values return `400` and name the field.
Documentation	None.	`openapi.yaml`, checked against the live API by `test_openapi.py`.
Client behavior	None.	`caller.py` with a timeout, retries with backoff, and a circuit breaker.

A3. OpenAPI: what it is and how it is used

`openapi.yaml` is an OpenAPI 3.0.3 description of the HTTP API. It lists every path and method, the request body, every response status, the response headers (`Location`, `Idempotent-Replayed`) and the JSON shape of each body (`CreateJobRequest`, `JobCreatedResponse`, `JobStatusResponse`, `ErrorResponse`). It does three jobs here:

- **A contract.** Anyone who writes a client can read one file instead of reading the Flask code.
- **A test.** `test_openapi.py` sends real requests to the running app and checks each answer against the schemas in the file, using `jsonschema`. If the code and the file drift apart, the test fails.
- **A starting point for tools.** Viewers such as Swagger UI, or client generators, can read the same file.

Result of `python test_openapi.py` on the running app:

```
(venv) PS D:\Habib\Sem 5\ESD\Homework_1\ESD-Assignment-1\job-queue-app> python test_openapi.py
============================================================
TEST 1: POST /jobs with a valid body -> expect 201,
        body matching JobCreatedResponse schema in openapi.yaml
============================================================
[PASS] Status code is 201
[PASS] 'Location' header present
[PASS] Response body matches JobCreatedResponse schema

============================================================
TEST 2: Repeat the exact same request (same Idempotency-Key)
        -> expect 200, same job_id, matching schema
============================================================
[PASS] Status code is 200 (idempotent replay)
[PASS] job_id matches the original request
[PASS] 'Idempotent-Replayed' header is 'true'

============================================================
TEST 3: POST /jobs with missing 'items' -> expect 400,
        body matching ErrorResponse schema in openapi.yaml
============================================================
[PASS] Status code is 400
[PASS] Response body matches ErrorResponse schema

============================================================
ALL CHECKS PASSED — live API responses match openapi.yaml
============================================================
```

Figure 1. python test_openapi.py: all checks pass against the running API.

A4. Idempotency

A client may send an `Idempotency-Key` header on `POST /jobs`. The key and the job it created are saved together in the `idempotency_keys` table. If the same key arrives again, the server returns the original job with status `200` and the header `Idempotent-Replayed: true`, and it does not make a new job. This is what makes retries safe: if a reply is lost and the client sends the request again, no duplicate invoice is made. The gRPC service uses the same rule through the same shared code.

A5. Rate limiting

`POST /jobs` allows 5 requests per 60 seconds for each client IP (a sliding window kept in memory). Above that, the API answers `429 Too Many Requests` with a `Retry-After` header. To test it without creating jobs, I sent 7 quick requests with an invalid body. The limiter runs before the body check, so the first 5 got through the limiter and were then rejected with `400`.

Table 2. Rate-limit test, 7 quick requests to `POST /jobs`.

Request	Response	Retry-After	Meaning
1 to 5	`400` invalid_request	none	Allowed by the limiter, then rejected by input checks (invalid body on purpose, so no jobs were made).
6 and 7	`429` rate_limited	`60`	Over the limit. Body: `Too many requests. Limit is 5 per 60 seconds. Try again in 60 seconds.`

Figure 2 shows the output of `python rate_limit_demo.py`.

```
(venv) PS D:\Habib\Sem 5\ESD\Homework_1\ESD-Assignment-1\job-queue-app> python rate_limit_demo.py
Sending 7 quick requests. The limit is 5 per 60 seconds per client.

request 1: HTTP 400
           Field 'items' is required and must be a non-empty list.
request 2: HTTP 400
           Field 'items' is required and must be a non-empty list.
request 3: HTTP 400
           Field 'items' is required and must be a non-empty list.
request 4: HTTP 400
           Field 'items' is required and must be a non-empty list.
request 5: HTTP 400
           Field 'items' is required and must be a non-empty list.
request 6: HTTP 429   Retry-After: 60 seconds
           Too many requests. Limit is 5 per 60 seconds. Try again in 60 seconds.
request 7: HTTP 429   Retry-After: 60 seconds
           Too many requests. Limit is 5 per 60 seconds. Try again in 60 seconds.

Done. The limiter counts requests per client, and a request is allowed
again once the oldest one leaves the 60 second window.
```

Figure 2. python rate_limit_demo.py: requests 1 to 5 get 400, requests 6 and 7 get 429 with Retry-After: 60.

`X-Skip-Rate-Limit: true` is a header used only by the test scripts, so that the retry demo is not blocked by the limiter. It would be removed in a real system.

A6. Python caller: timeout, retry and circuit breaker

`caller.py` sends `POST /jobs` with a 2 second timeout and up to 3 attempts. Between attempts it waits 1 second, then 2 seconds (exponential backoff). Every attempt of one submission uses the same `Idempotency-Key`, so a retry cannot make a duplicate. After 2 failed submissions in a row, the circuit breaker opens and blocks calls for 10 seconds without contacting the server. It then allows one trial call (half-open), and closes again if that call works. The script uses `/chaos` to cause each problem on purpose.

Table 3. Reliability test results (`python caller.py`).

Phase	What was injected	What the caller did	Result
1	Nothing.	1 attempt.	`201`, job created.
2	The server hangs once.	Attempt 1 timed out after 2 s. Waited 1 s. Attempt 2 used the same key.	`201`, job created once.
3a, 3b	The server fails (`500`) every time.	3 attempts each, waiting 1 s then 2 s. All returned `500`.	Both submissions failed. After the second one, the breaker opened.
3c	Breaker is open.	Blocked the call without contacting the server (10 s left).	Fast failure, no load on the server.

Phase	What was injected	What the caller did	Result
4	Wait 10 s, clear the faults.	Breaker went half-open and sent one trial call.	`201`. Breaker closed.

Figures 3 to 5 show the output of one run of `python caller.py`, in order.

```
(venv) PS D:\Habib\Sem 5\ESD\Homework_1\ESD-Assignment-1\job-queue-app> python caller.py
Caller settings: timeout=2s, max_attempts=3, backoff_base=1s, breaker_threshold=2 consecutive failures, breaker_recovery=10s

================================================================
PHASE 1: normal operation
================================================================
  attempt 1/3: POST /jobs (timeout=2s)
    -> success (201): {'job_id': '6d8c3c2c-58ac-477e-a2d5-7b139e7197aa', 'status': 'pending'}

================================================================
PHASE 2: server hangs once (timeout), then succeeds on retry
================================================================
  attempt 1/3: POST /jobs (timeout=2s)
    -> timed out after 2s
    -> retrying in 1s (safe: same Idempotency-Key: 1cd11572...)
  attempt 2/3: POST /jobs (timeout=2s)
    -> success (201): {'job_id': 'bcdce8fe-707c-459b-995a-b16369428df6', 'status': 'pending'}
```

Figure 3. python caller.py, phases 1 and 2: a normal call, then a timeout followed by a retry with the same key.

```
================================================================
PHASE 3: repeated server failures trip the circuit breaker open
================================================================
-- submission A (will exhaust retries and fail) --
  attempt 1/3: POST /jobs (timeout=2s)
    -> server error 500: {'error': {'code': 'internal_error', 'message': 'Simulated transient server failure (chaos: fail_next_n_requests).'}}
    -> retrying in 1s (safe: same Idempotency-Key: b35330c2...)
  attempt 2/3: POST /jobs (timeout=2s)
    -> server error 500: {'error': {'code': 'internal_error', 'message': 'Simulated transient server failure (chaos: fail_next_n_requests).'}}
    -> retrying in 2s (safe: same Idempotency-Key: b35330c2...)
  attempt 3/3: POST /jobs (timeout=2s)
    -> server error 500: {'error': {'code': 'internal_error', 'message': 'Simulated transient server failure (chaos: fail_next_n_requests).'}}
-- submission B (will exhaust retries, failure threshold reached, breaker opens) --
  attempt 1/3: POST /jobs (timeout=2s)
    -> server error 500: {'error': {'code': 'internal_error', 'message': 'Simulated transient server failure (chaos: fail_next_n_requests).'}}
    -> retrying in 1s (safe: same Idempotency-Key: 0c4ec5ed...)
  attempt 2/3: POST /jobs (timeout=2s)
    -> server error 500: {'error': {'code': 'internal_error', 'message': 'Simulated transient server failure (chaos: fail_next_n_requests).'}}
    -> retrying in 2s (safe: same Idempotency-Key: 0c4ec5ed...)
  attempt 3/3: POST /jobs (timeout=2s)
    -> server error 500: {'error': {'code': 'internal_error', 'message': 'Simulated transient server failure (chaos: fail_next_n_requests).'}}
  [breaker] 2 consecutive failed submissions -> opening circuit
-- submission C (breaker is now OPEN, should be blocked without calling the server) --
  [breaker] OPEN -> blocking call without contacting the server (10.0s left until a trial call is allowed)
```

Figure 4. python caller.py, phase 3: two submissions fail with 500, then the circuit breaker opens and blocks submission C.

```
================================================================
PHASE 4: waiting 10s for the breaker's recovery period, then clearing chaos and retrying
================================================================
  [breaker] recovery period elapsed -> HALF_OPEN, allowing one trial call
  attempt 1/3: POST /jobs (timeout=2s)
    -> success (201): {'job_id': 'fd8fab99-0881-4496-bfe9-396fe566a8c7', 'status': 'pending'}
  [breaker] trial call succeeded -> closing circuit

Done.
```

Figure 5. python caller.py, phase 4: after the recovery period one trial call succeeds and the breaker closes.

Part B: The gRPC Service

B1. Design

`grpc_server.py` is a separate program (port 50051). It offers one unary call, `SubmitJob`, which is the same operation as `POST /jobs`. It is not a wrapper around the HTTP endpoint. Both programs call the same functions in `job_store.py` (input checks, job creation, idempotency) and use the same SQLite file as the single source of truth. The background worker in `app.py` processes jobs from either path.

```
service JobService {
  rpc SubmitJob (SubmitJobRequest) returns (SubmitJobResponse);
}
message JobItem          { string name = 1; double qty = 2; double price = 3; }
message SubmitJobRequest { string customer = 1; repeated JobItem items = 2; string
idempotency_key = 3; }
message SubmitJobResponse{ string job_id = 1; string status = 2; bool idempotent_replayed =
3; }
```

Code generation command (only needed when `job_service.proto` changes):

```
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. job_service.proto
```

B2. Call paths

Figure 6 shows both paths. They start in different programs and meet at the shared business rules and the shared database. Logs from both programs go to the same file.

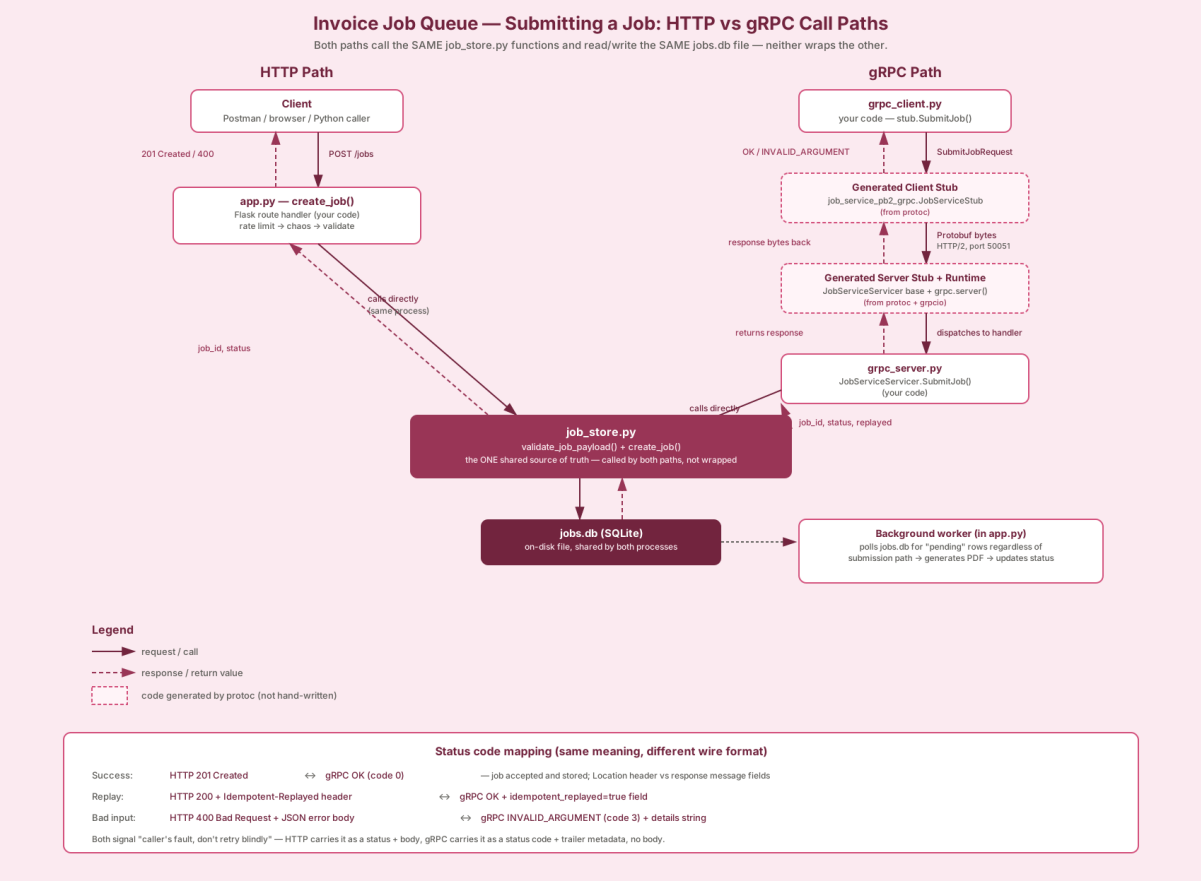


Figure 6. Call paths for HTTP and gRPC through the shared job store.

B3. Success and failure through both protocols

HTTP success (new job, Figure 7), then the same key again (replay, Figure 8):

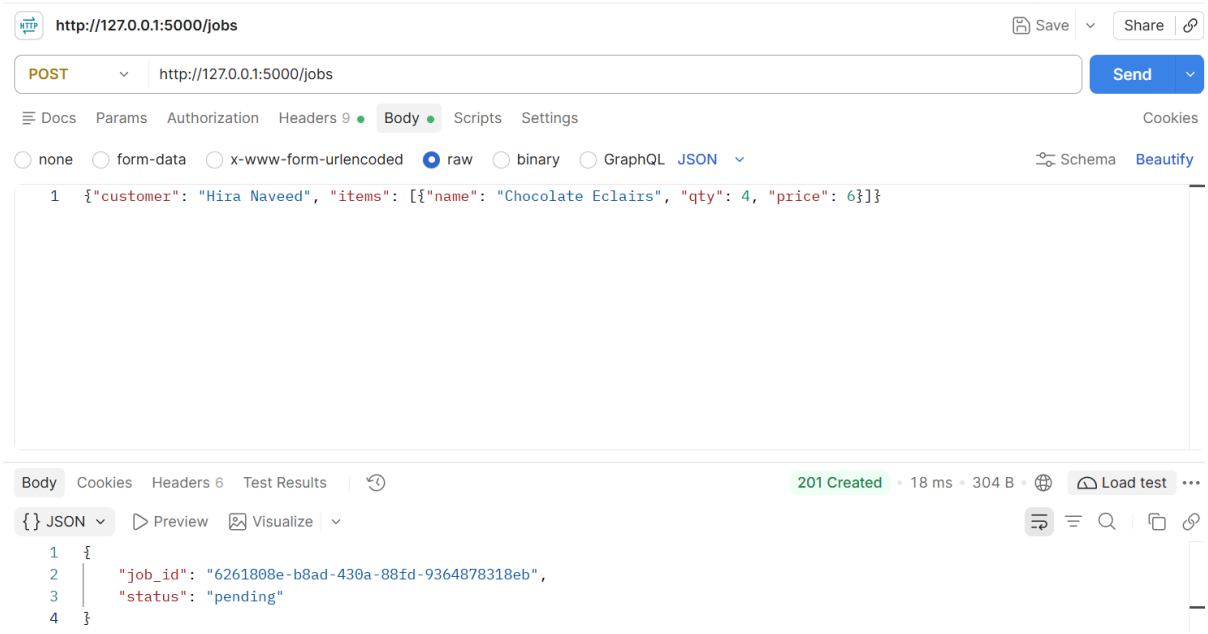
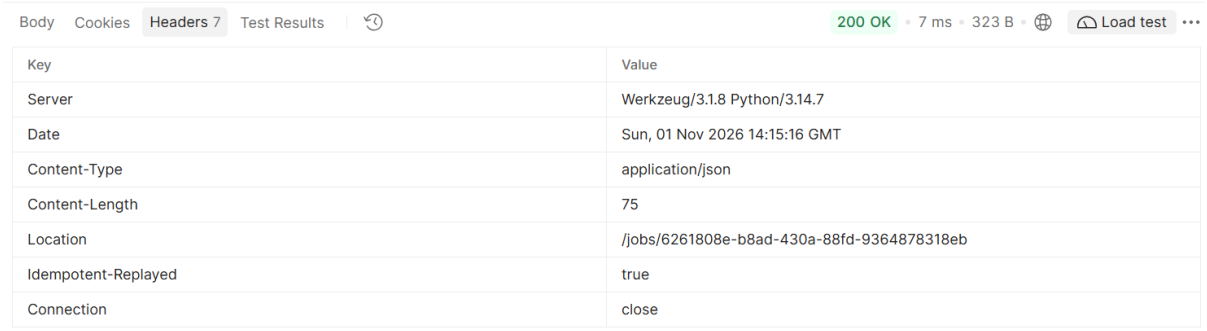


Figure 7. Postman: POST /jobs with an Idempotency-Key returns 201 Created and a job id.



Key	Value
Server	Werkzeug/3.1.8 Python/3.14.7
Date	Sun, 01 Nov 2026 14:15:16 GMT
Content-Type	application/json
Content-Length	75
Location	/jobs/6261808e-b8ad-430a-88fd-9364878318eb
Idempotent-Replayed	true
Connection	close

Figure 8. Postman: the same request again returns 200 OK, Idempotent-Replayed: true, and the Location of the original job.

HTTP failure (bad quantity, Figure 9, then an unknown job, Figure 10):

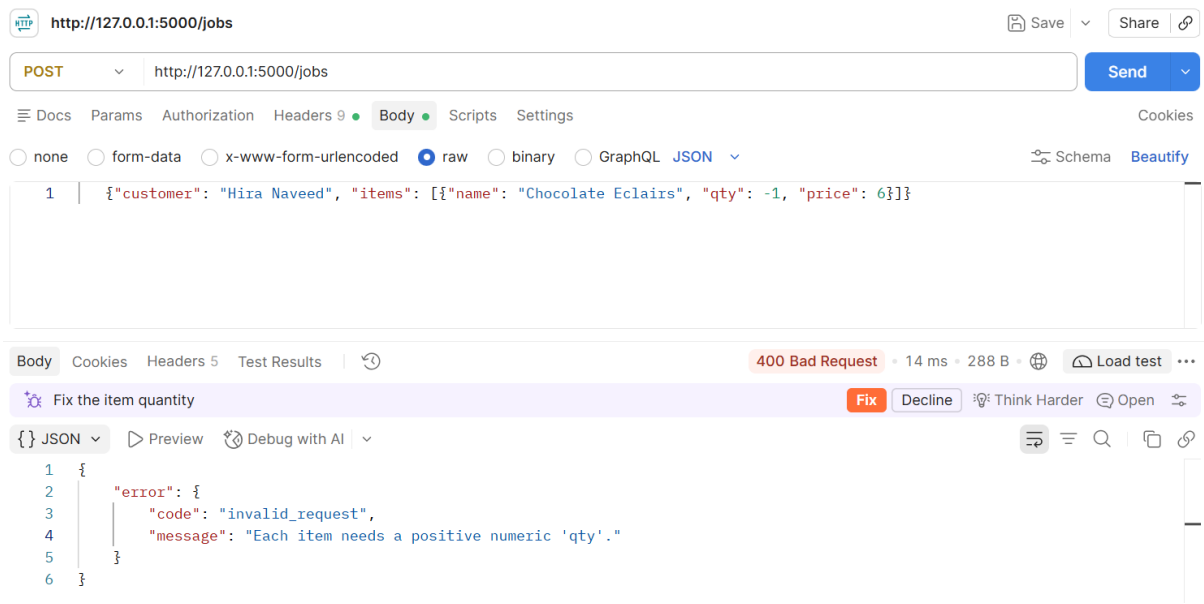


Figure 9. Postman: a negative quantity returns 400 Bad Request with the error envelope and a message that names the problem.

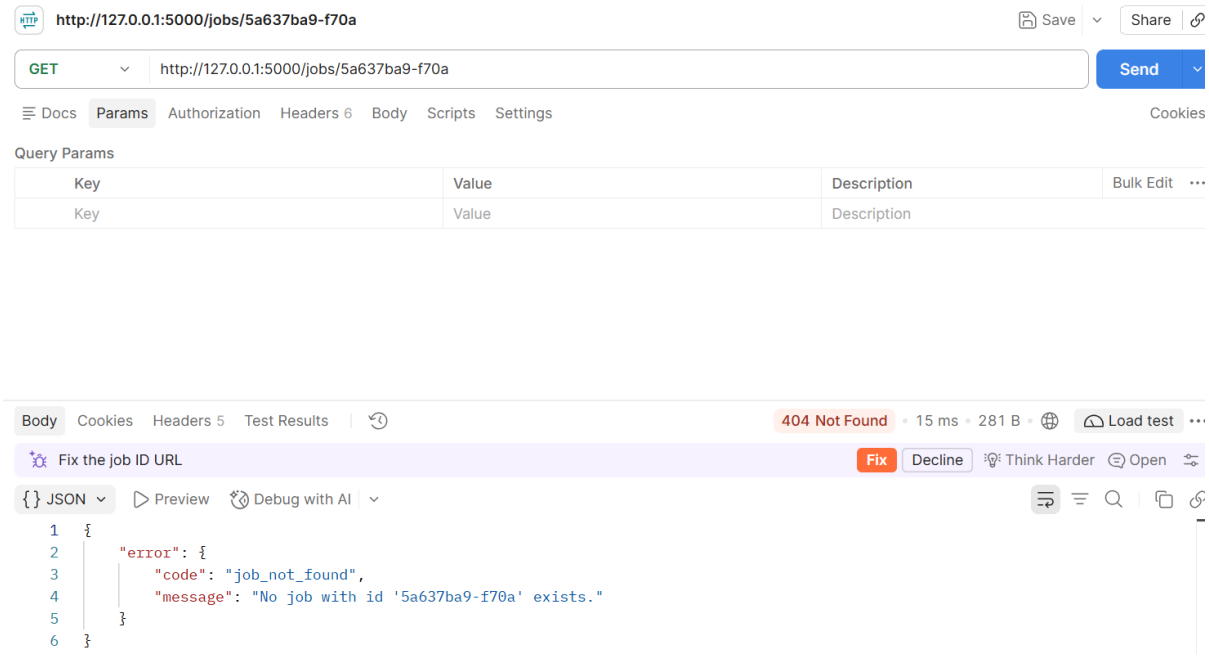


Figure 10. Postman: GET for an unknown job id returns 404 Not Found with the same envelope.

gRPC success and failure (`python grpc_client.py`, deadline 3 seconds, Figure 11):

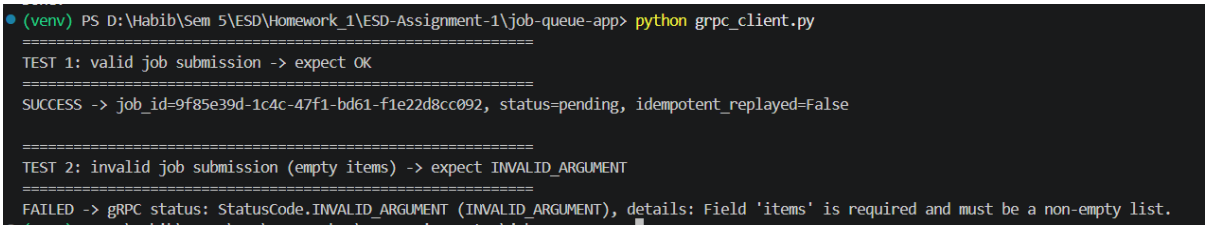


Figure 11. python grpc_client.py: a valid call returns OK with a job id; an empty items list returns INVALID_ARGUMENT.

The same input rule produces the same message on both paths: HTTP shows it as `400`, gRPC shows it as `INVALID_ARGUMENT`. This is because both call `job_store.validate_job_payload`.

B4. Same source of truth

A job created through gRPC can be read through HTTP, which shows both programs use one database. The job in Figure 11 has the id `9f85e39d-1c4c-47f1-bd61-f1e22d8cc092`. Figure 12 reads that id through the HTTP API:

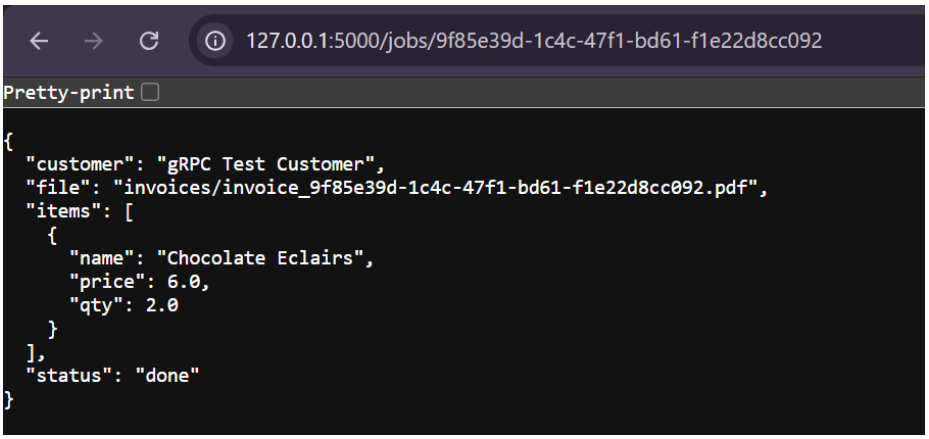


Figure 12. The job created through gRPC, read through the HTTP API (the address bar shows the same id).

B5. Logging and correlation are kept

Both programs write JSON logs to `logs/app.log` with the same `job_id` field. The field `app_service` says which program wrote the line. This is one job that came in over gRPC and was then processed by the worker in the HTTP program. Filtering Kibana by `job_id` shows the whole story:

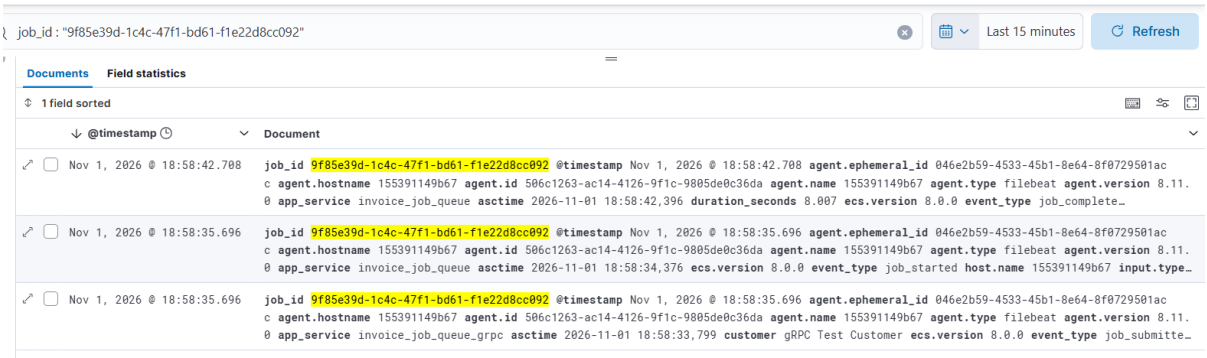


Figure 13. Kibana search for one job id: the submit line from the gRPC program, then the start and completion lines from the HTTP program.

Part C: Metrics

C1. Audit of the Assignment 1 metrics

Table 4. Which Assignment 1 metrics are still useful, what is missing, and what was done.

Metric	Verdict	Problem or gap	Action
`jobs_submitted_total`	Useful, incomplete	Counted only jobs sent over HTTP. Jobs sent over gRPC were invisible.	Fixed: added a `protocol` label (`http` or `grpc`), counted in both programs.
`jobs_in_progress`	Useful, flawed	It went up and down inside one process. With a second program adding jobs, the number would drift.	Fixed: the worker now sets it from a count in the shared database every cycle.
`job_processing_seconds`	Useful	None.	Kept.
`jobs_failed_total`	Useful	None.	Kept.
`invoice_file_size_kb`, `invoice_total_value`	Useful (business)	None.	Kept.
`cardinality_test_*`	Demo only	Not for real use.	Kept as the Assignment 1 demo.
HTTP requests and errors by endpoint	Missing	`400`, `404`, `429` and `500` answers were not counted anywhere.	Added `http_requests_total`.
HTTP latency	Missing	No view of how long requests take.	Added `http_request_duration_seconds`.
Rate-limit rejections	Missing	No direct count.	Added `rate_limit_rejections_total`.
Idempotent replays	Missing	No way to see how often retries hit the same key.	Added `idempotent_replays_total`.
gRPC calls, errors, latency, concurrency	Missing	No gRPC metrics at all.	Added `grpc_requests_total`, `grpc_request_duration_seconds`, `grpc_requests_in_flight`.

C2. New and changed metrics

The gRPC program has its own metrics page on port 8001 (`prometheus_client.start_http_server`), because each program has its own registry. Prometheus scrapes it as the job `invoice_job_queue_grpc` (`prometheus.yml`). The Python caller does not need a metrics endpoint.



Figure 14. Prometheus targets page: the HTTP app (port 5000) and the gRPC app (port 8001) are both UP.

Duration is measured on the server. The HTTP timer starts in `before_request` and stops in `after_request`. The gRPC timer wraps the `SubmitJob` handler and is observed in a `finally` block, so failed calls are timed too. Network time between client and server is not included. Percentiles are calculated over a 5 minute window.

Labels are always small, fixed sets: method, route template, status code, protocol. For HTTP, the label is the route template (`/jobs/<job_id>`), not the real URL, so job IDs never become label values. User, request and

trace IDs are never used as labels. Assignment 1's `/cardinality-test` shows why: one label value per request makes one new time series per request.

Table 5. New metrics (the first row changes an existing metric).

Metric and type	Purpose	Unit and labels	Code location	Query
`jobs_submitted_total` Counter (changed)	Count new jobs by how they arrived.	jobs `protocol`	`app.py` (create_job), `grpc_server.py` (SubmitJob)	`sum by (protocol) (increase(jobs_submitted_total{protoc`
`http_requests_total` Counter	See every HTTP answer, including errors and 429.	requests `method`, `endpoint`, `status`	`app.py` (after_request)	`sum by (endpoint, status) (rate(http_`
`http_request_duration_seconds` Histogram	Time to handle an HTTP request, on the server.	seconds `method`, `endpoint`	`app.py` (before_request, after_request)	`histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_b endpoint="/jobs"}[5m])))`
`rate_limit_rejections_total` Counter	Count requests refused by the rate limiter.	requests none	`app.py` (create_job)	`sum(increase(rate_limit_rejections_to`
`idempotent_replays_total` Counter	Count repeats answered from an existing key.	requests `protocol`	`app.py`, `grpc_server.py`	`sum by (protocol) (increase(idempotent_replays_total[5m`
`grpc_requests_total` Counter	Count gRPC calls by method and final status.	calls `method`, `code`	`grpc_server.py` (SubmitJob, finally block)	`sum by (method, code) (rate(grpc_requ`
`grpc_request_duration_seconds` Histogram	Time to handle a gRPC call, on the server.	seconds `method`	`grpc_server.py` (SubmitJob, finally block)	`histogram_quantile(0.95, sum by (le, (rate(grpc_request_duration_seconds_b`
`grpc_requests_in_flight` Gauge	Calls being handled right now (saturation).	calls `method`	`grpc_server.py` (inc at start, dec in finally)	`sum by (method) (grpc_requests_in_fli`

Buckets. The gRPC histogram uses 1 ms to 2.5 s (`0.001, 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5`) because local calls take a few milliseconds, so most of the buckets are near the bottom. The HTTP histogram adds 5 s and 10 s so that the 5 second delay from the chaos tests still lands in a real bucket.

Why in-flight and not message size. *The gRPC messages are small and always the same shape, so a size metric would show an almost flat line. Calls in flight shows saturation: it would rise if the handler slowed down, for example if the database were locked. With light load it stays near 0 and only shows a brief spike, because each call finishes in a few milliseconds and Prometheus samples every 5 seconds. That is the expected result, and the spike proves the metric is wired up.*

Other metrics added. The protocol label on `jobs_submitted_total`, the replay counter, and the rate-limit counter cover the project's own behavior that was missing: how jobs arrive, how often retries are safe replays,

and how often clients are slowed down. No further metric is needed, because job duration, failures, invoice size and invoice value are already covered from Assignment 1.

C3. Repeatable success and failure traffic

`python generate_traffic.py 60` sends the same fixed mix every time (60 rounds, about 1 minute), to both the HTTP API and the gRPC service, so the panels have both good and bad calls.

Table 6. What the traffic script sends.

When	HTTP	gRPC
Every round	`GET /jobs/<unknown id>`: `404`	Invalid call (empty items): `INVALID_ARGUMENT`
Every 3rd round	`POST /jobs` with a bad body: `400`	none
Every 5th round	Valid `POST /jobs`: `201`	Valid call: `OK`
Every 10th round	Same idempotency key: first `201`, then `200` (replay)	Same idempotency key: first creates, then replays
Rounds 15 and 45	Chaos: next 2 requests fail with `500`	none
Round 25	Burst of 7 requests without the skip header: five `400`, then `429`	none

C4. Dashboard evidence

The Grafana dashboard `Assignment 2 - HTTP vs gRPC` (exported as `grafana_dashboard_a2.json`) puts each gRPC panel next to its HTTP partner. The figures below were taken after running `python generate_traffic.py 120` (120 rounds, about 2 minutes).



Figure 15. Calls per second (top) and error rate (bottom). gRPC on the left, HTTP on the right.

Both sides show the same shape: traffic rises while the script runs and falls back to zero when it ends. On the HTTP side the `400`, `404`, `201`, `200`, `429` and `500` answers are all visible as separate lines. For gRPC, `INVALID_ARGUMENT` is the biggest line, at about 1 call per second, and `OK` is about 0.3 calls per second.

The error rates are high on purpose, because the script sends mostly bad calls. Between about 75% and 80% of gRPC calls were `INVALID_ARGUMENT` while the script ran. For HTTP, `/jobs/<job_id>` is at 100% (every call asks for an unknown job) and `/jobs` is between about 50% and 68%.



Figure 16. Latency p50 and p95 (top), gRPC calls in flight (bottom left), and jobs submitted, replays and rate-limit rejections (bottom right).

Latency. p50 is under 1 ms on both protocols. p95 is about 20 ms for gRPC and about 22 ms for HTTP. A percentile from a histogram is an estimate inside a bucket, so these are close rather than exact. For this operation the two protocols are about the same speed, because most of the time goes into the shared database work.

In flight. The gauge sits at 0 most of the time and shows one short spike to 1 near the start. Each call finishes in a few milliseconds and Prometheus samples every 5 seconds, so under this load the gauge is rarely caught above 0.

Jobs, replays and rejections. HTTP and gRPC each created about 25 jobs. There were about 11 replays over HTTP and about 10 over gRPC, and 2 rate-limit rejections, which match the two `429` answers in the burst at round 25.

C5. Dashboard queries

Panel	Query
gRPC calls per second by method and status	`sum by (method, code) (rate(grpc_requests_total[1m]))`
HTTP calls per second by endpoint and status	`sum by (endpoint, status) (rate(http_requests_total[1m]))`
gRPC error rate by method	`sum by (method) (rate(grpc_requests_total{code!="OK"}[1m])) / sum by (method) (rate(grpc_requests_total[1m]))`
HTTP error rate by endpoint	`sum by (endpoint) (rate(http_requests_total{status=~"[45].."}[1m])) / sum by (endpoint) (rate(http_requests_total[1m]))`
gRPC latency p50 / p95 (5m)	`histogram_quantile(0.50, sum by (le, method) (rate(grpc_request_duration_seconds_bucket[5m])))` and the same with `0.95`
HTTP POST /jobs latency p50 / p95 (5m)	`histogram_quantile(0.50, sum by (le) (rate(http_request_duration_seconds_bucket{method="POST", endpoint="/jobs"}[5m])))` and the same with `0.95`
gRPC calls in flight	`sum by (method) (grpc_requests_in_flight)`
Jobs submitted, replays, rate-limit rejections	`sum by (protocol) (increase(jobs_submitted_total{protocol!=""}[5m])), sum(increase(rate_limit_rejections_total[5m])), sum by (protocol) (increase(idempotent_replays_total[5m]))`

What Is Submitted and How to Reproduce

Item	File
Code	`app.py`, `grpc_server.py`, `job_store.py`
OpenAPI document	`openapi.yaml`
.proto and generation command	`job_service.proto` (command in Part B and in `README.md`)
Python callers	`caller.py` (HTTP), `grpc_client.py` (gRPC)
Tests	`test_openapi.py`, `generate_traffic.py`, `test_pdf.py`
Prometheus config	`prometheus.yml` (scrapes the HTTP app, the gRPC app and the machine exporters)
Grafana dashboard	`grafana_dashboard_a2.json`
README with run and test commands	`README.md`

To reproduce: start `python app.py` and `python grpc_server.py` in two terminals, then run `python test_openapi.py`, `python caller.py`, `python grpc_client.py` and `python generate_traffic.py 60`, and open the Grafana dashboard with the time range set to the last 15 minutes.